



Formation Spring Boot



Gestion par configuration
(Convention Over Configuration)



Spring Boot

2

- ✓ Objectif
- ✓ Historique
- ✓ Fonctionnement
- ✓ Docker & Spring Boot
- ✓ GraalVM et Spring Boot
- ✓ Les profiles
- ✓ Mise en place Maven
- ✓ Classe de lancement
- ✓ Fichier de paramétrage
- ✓ Tester avec Spring Boot
- ✓ Changer une version dans mes dépendances
- ✓ Spring Boot et les annotations conditionnelles



Objectifs Spring Boot

- Les grosses problématiques du Spring (2, 3, 4)
 - **La gestion du MAVEN**, les dépendances et leurs versions
 - ❑ Un vrai casse-tête, l'évolution d'une librairie ou une erreur de version rend le projet caduc
 - ❑ Hors, on ne peut pas savoir avec certitude la bonne version à prendre avec tel ou tel version de Spring
 - **La gestion des objets techniques** du Spring
 - ❑ Data Source, transactions, pool, sécurité, Web Services ...
 - ❑ On ne connaît pas toujours le nom des classes ou la manière de bien les configurer
 - **Une configuration parfois complexe**
 - ❑ Via classes ou XML voir les deux, mais assez lourde.



Objectifs Spring Boot

- **Spring Boot** permet de résoudre les trois problématiques principales suivante :
 - La gestion du MAVEN, elle reste minimaliste, Spring boot s'occupe de 'savoir' les bonnes versions qui sont adaptées à la version du Spring
 - La gestion des objets techniques du Spring
 - ❑ Plus de classes ou de fichiers XML à réaliser, les objets vont se fabriquer 'tout seul' en fonction des propriétés déclarées dans un fichier de configuration
 - Dans les cas simples, la configuration se limitera à une seule classe principale.
 - ❑ Plus d'XML, restera uniquement des annotations pour faire de l'auto-configuration



Objectifs Spring Boot

➤ Mais Spring Boot permet aussi :

➤ De simplifier sa mécanique de tests

- ❑ Possibilité d'auto embarquer un 'mini' serveur JEE pour tester son application
- ❑ Plus besoin d'un serveur JEE ou même d'une base de données

➤ De fabriquer une image docker du projet (sans ses dépendances)

- ❑ Pas besoin de DockerFile, SB se charge de s'auto fabriquer son image docker et vous la met à disposition dans votre Docker local

➤ De lancer son/ses dockerfile automatiquement au démarrage du projet java

- ❑ Dans cette approche c'est le projet SB qui va démarrer automatiquement les autres conteneurs ET va se brancher dessus
 - Depuis SB 3.1 : pour le moment ne supporte pas tous les processus externes



Historique

- Spring Boot existe depuis Spring 4
 - Dernière version **1.5.22** (08/2019)
- La version **2** de Spring Boot a vu le jour avec la sortie de Spring 5
 - Dernière version (Maven) 2.7.18 (11/2023)
- Spring Boot **2.5** supporte :
 - Supporte Java 16, Kotlin 1.4
 - Gradle 7
 - Docker image
 - Spring WebFlux
 - NoSQL : Cassandra, MongoDB, Couchbase, Redis
 - HTTP/2 pour Tomcat/UnderTow/Jetty
 - Supporte Gradle 4.x
 - JUnit 5



Historique

- Spring Boot **3.x.y** (arrivée fin novembre 2022)
 - Ne supportera **PLUS** Java 8 (ou 11)
 - Le support commencera **à partir** de Java **17**
 - Nous serons en Spring **6.x**
 - Support minimal de JEE 9 ⇔ Jakarta EE 9
 - ❑ Servlet 5 et JPA 3
 - ❑ javax.servlet. ⇔ jakarta.servlet.
 - ❑ javax.persistence ⇔ jakarta.persistence.



Historique

- Spring Boot 4.x.y (arrivée fin novembre 2025)
 - Le support commencera à partir de Java 17 et va jusqu'au Java 21
 - Nous serons en Spring 7.x
 - Support minimal de JEE 11 ⇔ Jakarta EE 11
 - Gros changement dans la mécanique des Spring Boot Starter
 - Changement de version de Jackson (bibliothèque qui gère le JSON)



Historique

- Spring Boot 4.1.y (arrivée Juin 2026)
 - Ajout du gRPC (partie serveur et client + tests)
 - Ajout du SSRF (Server Side Request Forgery)
 - Gestion native des log rotatif de Log4J
 - Amélioration de la gestion de Kafka
 - Amélioration du Spring Data (prise en compte de Redis)



Versions de Spring Boot

- **ATTENTION** : à votre **version** de Spring Boot
 - C'est le 'pilote' dans l'avion, vous êtes le co-pilote
- Ne **JAMAIS** changer de version 'on the fly'
- Lisez **TOUJOURS** les release notes quand vous voulez changer de version mineure et **SURTOUT** majeure
- Spring Boot est votre ami, mais lors d'une migration les changements de dépendances peuvent casser votre projet
 - Dans le meilleur des cas, il ne compile plus
 - Dans le pire des cas, il marchera à moitié



Fonctionnement

- C'est Spring Boot qui prend la main sur votre **MAVEN**
 - Il supporte aussi Gradle
- Dans le cas de Maven, votre fichier pom.xml
 - **Hériter**a d'un POM parent géré par Spring
 - Contiendra des dépendances vers Spring Boot et ses plugins / starter
 - Évitera au maximum les numéros de versions (elles sont toutes dans le pom parent = celui du Spring Boot).
 - Deviendra vraiment plus simple



Fonctionnement

➤ Au niveau du code :

➤ Une classe Java qui portera l'auto-configuration du projet

❑ Elle portera l'annotation **@SpringBootApplication**

○ Elle sera **la seule** à porter cette annotation

○ Elle devra **se trouver dans votre package racine**

❑ Elle s'occupera d'analyser

○ Votre code Java ($\Leftrightarrow$ le component scan)

○ Vos dépendances ($\Leftrightarrow$ les librairies dans votre classpath)

➤ Au moins un fichier **application.properties** (ou *.yml*)

❑ Contiendra des directives de configuration pour Spring Boot



Fonctionnement

- Structure de vos packages
 - Vous **DEVEZ** avoir un package racine
 - ❑ com.banque
 - Tous les autres packages seront enfants du package racine
 - ❑ com.banque.repository
 - ❑ com.banque.service
 - ❑ com.banque.controler
 - ❑ ...
- Votre classe @SpringBootApplication **devra** être au niveau de votre package racine
 - com.banque (par rapport à l'exemple ci-dessus)
 - Idem pour la classe de configuration des tests unitaires



Maven

14

➤ Un Maven Simple pour Spring Boot

```
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
```

<!-- En Spring Boot votre parent est toujours Spring Boot -->

<parent>

<groupId>org.springframework.boot</groupId>

<artifactId>spring-boot-starter-parent</artifactId>

<!-- Version de Spring Boot a utiliser (Nous serons en Spring 4 ici) -->

<version>1.5.22.RELEASE</version>

<relativePath /> <!-- lookup -->

</parent>

<modelVersion>4.0.0</modelVersion>

<groupId>mon.groupe.id</groupId>

<artifactId>mon.artefact.id</artifactId>

1.5.xx <=> Spring 4
2.y.xx <=> Spring 5
3.y.xx <=> Spring 6
4.y.xx <=> Spring 7

Votre projet



spring-boot-maven-plugin

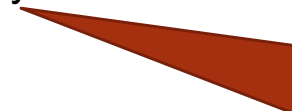
15

- Un plugin pour Maven
 - Objectif : fabriquer un JAR / WAR **auto-exécutable**
- Permet de
 - lancer son application via un goal Maven
 - ❑ mvn spring-boot:run
 - Lancer les tests d'intégration avant l'application
 - java -jar monLivrable.xxx
- Le plugin n'est pas obligatoire, il ne sert que dans le cas de l'intégration continue

```
<build>

  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
      <configuration>
        <mainClass>${start-class}</mainClass>
        <layout>JAR</layout>
      </configuration>
      <executions>
        <execution>
          <goals>
            <goal>repackage</goal>
          </goals>
        </execution>
      </executions>
    </plugin>
  </plugins>

</build>
```



JAR
WAR
ZIP
MODULE
NONE



spring-boot-maven-plugin

➤ Les Goals maven associés

- Pour lancer le projet (en mode bloquant)

```
mvn spring-boot:run
```

- Pour lancer / arrêter le projet (en mode non bloquant)

- ☐ En principe, sert pour les TI avec mvn verify

```
mvn spring-boot:start
```

```
mvn spring-boot:stop
```

- Pour fabriquer une image docker

```
mvn spring-boot:build-image
```




spring-boot-maven-plugin

➤ Start et stop permettent de lancer des TI sans bloquer de terminal

- Il faut cependant écrire quelques lignes supplémentaires
- Le Goal ***mvn verify*** enclenchera votre maven-failsafe-plugin qui lancera les XxxIT classes

```
<plugin>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-maven-plugin</artifactId>
  <executions>
    <execution>
      <id>pre-integration-test</id>
      <goals>
        <goal>start</goal>
      </goals>
    </execution>
    <execution>
      <id>post-integration-test</id>
      <goals>
        <goal>stop</goal>
      </goals>
    </execution>
  </executions>
</plugin>
```

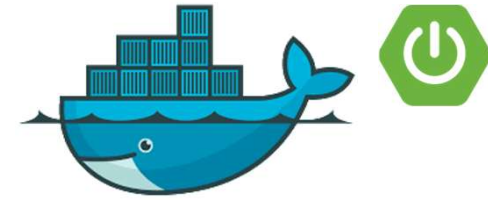


spring-boot-maven-plugin

- N'oubliez pas **deux points** importants pour vos TI
 - Le nom des classes doit se terminer par **IT** (IntegrationTest à l'Anglaise) et pas Test
 - ❑ ControleurZqsdqsd**IT.java**
 - Vous devez inclure le plugin maven-failsafe-plugin

```
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-failsafe-plugin</artifactId>
  <executions>
    <execution>
      <goals>
        <goal>integration-test</goal>
        <goal>verify</goal>
      </goals>
    </execution>
  </executions>
</plugin>
```

spring-boot-maven-plugin



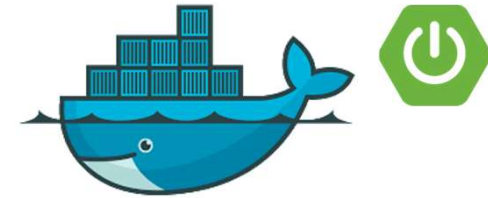
- Vous pouvez aussi vous en servir pour fabriquer une image docker à la volée
- Il faut un démon docker en local sur votre machine

```
mvn spring-boot:build-image -Dspring-boot.build-image.imageName=myorg/myapp
```

- Puis lancez votre image (ici myorg/myapp)

```
docker run -p 8080:8080 docker.io/myorg/myapp:latest
```

spring-boot-maven-plugin



- Vous pouvez aussi faire passer des informations directement dans votre pom.xml

```
<plugin>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-maven-plugin</artifactId>
  <configuration>
    <image>
      <network>host</network>

      <env>
        <HTTP_PROXY>http://mon-proxy.entreprise.com:8080</HTTP_PROXY>
        <HTTPS_PROXY>http://mon-proxy.entreprise.com:8080</HTTPS_PROXY>
        <NO_PROXY>localhost,127.0.0.1,.mon-entreprise.internal</NO_PROXY>
      </env>
    </image>
  </configuration>
</plugin>
```

native-maven-plugin



- Depuis la version 3 de Spring Boot vous pouvez réaliser des images **Java Native** de votre application
 - Spring Boot fabriquera une image Docker de
 - ❑ Votre application en Java Native + **GraalVM**
- **Attention** : cette technologie n'est pas encore 100% compatible avec l'ensemble des éléments du Spring et du Java
- Avantages : la GraalVM permet de gagner en consommation mémoire et temps d'utilisation du processeur
 - Si vous déployer sur un cloud <=> gains monétaires importants

native-maven-plugin



➤ Limitations

➤ Pas de chargement dynamique des beans

- ❑ Limitation sur l'utilisation des `@Profile` et annotations conditionnelles
- ❑ Idéalement, désactiver le proxy
 - `@Configuration(proxyBeanMethods=false)`

➤ Attention aux versions de certaines librairies

- ❑ Log4J, JWT ... : Peuvent fonctionner, mais pas toutes les versions
- ❑ Pas toujours évident de s'y retrouver
- ❑ De nombreux bug avec Docker sous Windows

native-maven-plugin



➤ Etape 1

- Installez sur votre machine une GraalVM

- ❑ <https://www.oracle.com/downloads/graalvm-downloads.html>

- Ajoutez la variable d'environnement

- ❑ GRAALVM_HOME=c:\toto\myGraalVMDir

- Dans votre pom.xml, ajoutez la dépendance vers le plugin (exemple simple)

```
<build>
  <plugins>
    <plugin>
      <groupId>org.graalvm.buildtools</groupId>
      <artifactId>native-maven-plugin</artifactId>
    </plugin>
  </plugins>
</build>
```

native-maven-plugin



```
<plugin>
  <groupId>org.graalvm.buildtools</groupId>
  <artifactId>native-maven-plugin</artifactId>
  <extensions>true</extensions>
  <executions>
    <execution>
      <id>build-native</id>
      <goals>
        <goal>compile</goal>
      </goals>
      <phase>package</phase>
    </execution>
  </executions>
  <configuration>
    <metadataRepository>
      <enabled>true</enabled>
    </metadataRepository>
    <buildArgs>
      <buildArg>--initialize-at-build-time=org.hibernate.validator.internal.util.Version</buildArg>
      <buildArg>-H:PartialEscapeAnalysis=false</buildArg>
    </buildArgs>
  </configuration>
</plugin>
```


native-maven-plugin



➤ Etape 2

- Vous devez avoir un démon Docker d'installé sur votre machine de build
- Idéalement, en rootless

➤ Etape 3

- Lancez la commande

```
mvn -Pnative spring-boot:build-image -Dmaven.test.skip=true
```

- ❑ C'est la même commande que pour l'image traditionnelle mais le -Pnative fera usage du **Profile Maven native** inclus dans votre POM parent (hérité du Spring Boot)

native-maven-plugin



➤ Etape 4

- Si la build s'est bien déroulée, votre image docker a été créée et est disponible sous le nom indiqué en fin de build

❑ /chemin/nomDuProjet:version

```
[INFO]
[INFO] Successfully built image 'docker.io/library/stone.lunchtime:2.2.06'
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
```

➤ Etape Finale

- Lancez votre conteneur

```
docker run --rm -p 8080:8080 /chemin/nomDuProjet:version
```

native-maven-plugin



- Pas encore bien fiable, mais bientôt ...
 - Lisez attentivement les messages lors des étapes 4 et finale
 - ❑ Warning: Could not resolve class ...
 - Très souvent, le premier lancement échouera
 - ❑ Il faudra évaluer la faisabilité du contournement
 - Ajustement du code
 - Attention aux caractères NON ASCII
 - Changement de version de librairie (car pas encore totalement compatible GraalVM)
 - Changement de la plateforme de build (sous Linux plutôt que sous Windows)
 - Passer en JAR plutôt que WAR

native-maven-plugin



- Gestion d'Hibernate, mais il faut tout de même faire attention
- https://docs.jboss.org/hibernate/orm/6.2/userguide/html_single/Hibernate_User_Guide.html#tooling-enhancement

```
<plugin>
  <groupId>org.hibernate.orm.tooling</groupId>
  <artifactId>hibernate-enhance-maven-plugin</artifactId>
  <version>${hibernate.version}</version>
  <executions>
    <execution>
      <configuration>
        <failOnError>true</failOnError>
        <enableLazyInitialization>true</enableLazyInitialization>
      </configuration>
      <goals>
        <goal>enhance</goal>
      </goals>
    </execution>
  </executions>
</plugin>
```

native-maven-plugin



- En version > 6 d'Hibernate, il est préférable d'utiliser

```
<plugin>
  <groupId>org.hibernate.orm</groupId>
  <artifactId>hibernate-maven-plugin</artifactId>
  <version>${hibernate.version}</version>
  <executions>
    <execution>
      <id>enhance</id>
      <goals>
        <goal>enhance</goal>
      </goals>
      <configuration>
        <enableLazyInitialization>true</enableLazyInitialization>
        <enableDirtyTracking>true</enableDirtyTracking>
        <enableAssociationManagement>false</enableAssociationManagement>
        <enableExtendedEnhancement>false</enableExtendedEnhancement>
      </configuration>
    </execution>
  </executions>
</plugin>
```

native-maven-plugin

GraalVM.



30

- Attention aux versions de vos librairies externes
 - Pour chaque librairie, ou presque, on peut trouver une configuration pour GraalVM.
 - On trouve ses fichiers de configuration dans le dossier `target/graalvm-reachability-metadata\xxx\groupId\artifactId`
- Pour les librairies qui posent problème, **vérifiez les versions** indiquées dans ces dossiers
 - Potentiellement, réajustez vos versions dans votre POM afin de matcher ces dernières

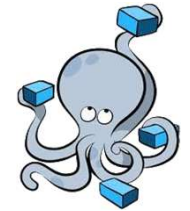
```
target
├── generated-sources
├── graalvm-reachability-metadata
│   └── 73921877cd7b58d9093c9587c83d6a3fdcacda95
│       ├── ch.qos.logback
│       ├── com.ecwid.consul
│       ├── com.fasterxml.jackson.core
│       ├── com.github.ben-manes.caffeine
│       ├── com.github.luben
│       ├── com.google.protobuf
│       ├── com.graphql-java
│       ├── com.h2database
│       ├── com.hazelcast
│       ├── com.microsoft.sqlserver
│       ├── com.mysql
│       ├── com.sun.mail
│       ├── com.zaxxer
│       ├── commons-logging
│       ├── io.grpc
│       ├── io.jsonwebtoken
│       ├── io.nats
│       ├── io.netty
│       ├── io.opentelemetry
│       ├── io.undertow
│       ├── jakarta.servlet
│       ├── javax.cache
│       └── mysql
```

native-maven-plugin



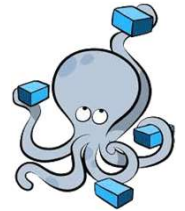
- Pour trouver des solutions quand cela ne fonctionne pas
 - Faites-vous une VM Linux (si vous êtes sous Windows)
 - Installez sur votre machine Linux
 - ☐ Docker
 - ☐ GraalVM
 - ☐ GCC
 - ☐ Maven
 - ☐ ZIP
 - ☐ zlib1g-dev
 - Plutôt que de lancer le goal de fabrication d'image, lancez celui de compilation
 - ☐ `mvn -Pnative clean native:compile`
 - ☐ Les messages sont plus explicites

spring-boot-docker-compose (v3.1+)



- Vous pouvez aussi vous en servir pour démarrer automatiquement vos fichiers docker-compose à la volée
 - <https://docs.spring.io/spring-boot/reference/features/dev-services.html#features.dev-services.docker-compose>
- Dans cette approche,
 - Je lance mon Spring Boot
 - Il va se charger de piloter docker en lançant le/les fichiers docker-compose spécifiés
 - ET : votre Spring Boot va regarder dans le dockerfile pour SAVOIR le/les URLs, Login, pwd ... nécessaires pour que le Java puisse accéder au service conteneurisé
 - ❑ Plus besoin d'indiquer dans le application.properties l'url/login/pwd de votre SGBD
- Pratique quand vous voulez vous assurer que l'ensemble des conteneurs dont dépendent votre projet sont lancés

spring-boot-docker-compose (v3.1+)



- Vous devrez ajouter la dépendance maven

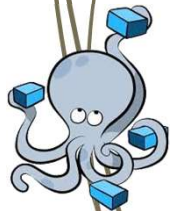
```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-docker-compose</artifactId>
  <optional>true</optional>
</dependency>
```

- Préparez votre fichier docker compose.yaml
 - Comme vous avez l'habitude
 - **! ATTENTION** : il ne doit pas enclencher la build de votre projet Java
 - Selon le nom de votre service, vous devrez potentiellement y ajouter un **label** afin que SB le trouve correctement

spring-boot-docker-compose (v3.1+)

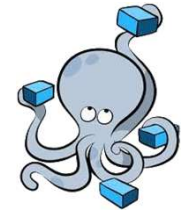
➤ Exemple d'un docker compose pour MySQL

- Notez l'ajout de **labels**, c'est grâce à cette ligne que SB va trouver notre base MySQL et les informations qui lui sont liées



```
name: stone-lunchtime-project-mysql
services:
  mysql_sl_db:
    image: mysql:8.0.32
    restart: unless-stopped
    env_file: ./env
    container_name: mysql_sl_db
    hostname: mysql_sl_db
    labels:
      org.springframework.boot.service-connection: mysql
    environment:
      - TZ=$TIME_ZONE
      - MYSQL_ROOT_PASSWORD=$DB_ROOT_PASSWORD
      - MYSQL_DATABASE=$DB_DATABASE
      - MYSQL_USER=$DB_USER
      - MYSQL_PASSWORD=$DB_USER_PASSWORD
      - MYSQL_ALLOW_EMPTY_PASSWORD=no
    ports:
      - $DB_LOCAL_PORT:$DB_DOCKER_PORT
    volumes:
      - type: bind
        source: ./dump/insert.sql
        target: /docker-entrypoint-initdb.d/insert.sql
        read_only: true
      - type: volume
        source: vol_mysql_sl_db
        target: /var/lib/mysql
    volumes:
      vol_mysql_sl_db:
```

spring-boot-docker-compose (v3.1+)



➤ Exemple d'un fichier properties

➡ Notez

- ❑ l'ajout de **spring.docker.compose.file** qui pointe vers le fichier *compose.yaml* qui s'occupe de lancer MySQL
- ❑ il n'y a **PAS** les clefs pour l'URL, le login, le mot de passe d'accès à la base de données

```
spring.jpa.properties.hibernate.dialect.storage_engine=innodb
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQLDialect
spring.jpa.properties.hibernate.globally_quoted_identifiers=true

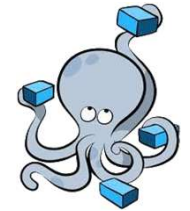
spring.docker.compose.file=./docker/mysql/compose.yaml
```

spring-boot-docker-compose (v3.1+)



- Quand vous allez démarrer votre projet Spring Boot, il va automatiquement
 - vous lancer votre *compose.yaml* (comme si vous aviez tapé *docker compose up*)
 - faire le lien entre vos processus dockerisés et votre application Java, à ce jour seuls les conteneurs suivants sont supportés (le SSL n'est pas disponible pour tous);
 - ❑ "symptoma/activemq", "apache/activemq-classic", "apache/activemq", "apache/activemq-artemis", "apache/artemis", "rabbitmq"
 - ❑ "cassandra"
 - ❑ "elasticsearch", "elasticsearch/elasticsearch"
 - ❑ "hazelcast/hazelcast".
 - ❑ "clickhouse/clickhouse-server", "gvenzl/oracle-free", "gvenzl/oracle-xe", "mariadb", "mssql/server", "mysql", "postgres"
 - ❑ "osixia/openldap", "ldap/ldap"
 - ❑ "mongo"
 - ❑ "neo4j"
 - ❑ "otel/opentelemetry-collector-contrib", "grafana/otel-lgtm"
 - ❑ "apache/pulsar/pulsar"
 - ❑ "clickhouse/clickhouse-server", "gvenzl/oracle-free", "gvenzl/oracle-xe", "mariadb", "mssql/server", "mysql", "postgres"
 - ❑ "redis", "redis/redis-stack", "redis/redis-stack-server"
 - ❑ "openzipkin/zipkin".

spring-boot-docker-compose (v3.1+)



- Vous pouvez indiquer un nom de profil (ou plusieurs) qui enclencheront le docker-profile
 - Pratique si vous ne voulez pas qu'il se lance en production par exemple

```
spring.docker.compose.profiles.active=monprofilequidoitlancerdockercompose
```

- Vous pouvez indiquer des options de pilotage du docker compose

```
spring.docker.compose.readiness.tcp.connect-timeout=10s  
spring.docker.compose.readiness.tcp.read-timeout=5s  
# Ou none, start-only plutôt que start-and-stop  
spring.docker.compose.lifecycle-management=start-and-stop  
spring.docker.compose.start.command=start  
spring.docker.compose.stop.command=down  
spring.docker.compose.stop.timeout=1m
```



DevTools

- **Devtools** regroupe des outils pour simplifier le développement
 - Remontage à chaud du code ou des ressources en cas de modifications

```
<dependencies>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-devtools</artifactId>
  <optional>true</optional>
</dependency>
</dependencies>
```

- Cette dépendance n'est pas obligatoire, elle sert surtout dans le cas où vous faites usage du plugin spring-boot-maven-plugin en mode Web (ou d'un lancement via docker)
 - Evite d'avoir à arrêter/relancer son projet à chaque changement de code



DevTools

39

- Par défaut, devtools ne se lancera pas si vous lancez votre projet en mode *java -jar* (~mode production)

- C'est pour des raisons de sécurité
- Si vous voulez le changer :
`spring.devtools.restart.enabled=true`

- L'utilisation du devtools appliquera automatiquement les propriétés de configuration suivantes

- Si vous ne le souhaitez pas :
`spring.devtools.add-properties=false`



```
management.tracing.sampling.probability=1.0
server.servlet.jsp.init-parameters.development=true
server.servlet.session.persistent=true
spring.docker.compose.readiness.wait=only-if-started
spring.freemarker.cache=false
spring.graphql.graphiql.enabled=true
spring.groovy.template.cache=false
spring.h2.console.enabled=true
spring.mustache.servlet.cache=false
spring.mvc.log-resolved-exception=true
spring.reactor.netty.shutdown-quiet-period=0s
spring.template.provider.cache=false
spring.thymeleaf.cache=false
spring.web.error.include-binding-errors=always
spring.web.error.include-message=always
spring.web.error.include-stacktrace=always
spring.web.resources.cache.period=0
spring.web.resources.chain.cache=false
```



DevTools

- Par défaut, devtools relancera votre projet automatiquement dès qu'un changement dans le classpath est détecté
 - Vous pouvez inclure d'autres éléments
 - ☐ `spring.devtools.restart.additional-paths`
 - Vous pouvez exclure des éléments :
 - ☐ `spring.devtools.restart.exclude=static/**,public/**`
 - ☐ `spring.devtools.restart.additional-exclude=xxxxx`
 - Vous pouvez indiquer un fichier, qui si il est modifié, va enclencher le relancement du projet
 - ☐ Exemple d'un fichier se trouvant dans `src/main/resources/.reloadtrigger`
 - `spring.devtools.restart.trigger-file=.reloadtrigger`



DevTools

- Toute la configuration spécifique à devtools peut être rangée dans un fichier
 - `spring-boot-devtools.properties` (ou `yaml` ou `yml`)
- Le devtool fonctionne en local mais peut aussi fonctionner en remote
 - Cette fonctionnalité n'est pas disponible dans cas des applications webflux
 - Ne pas activer cette option en production pour des raisons de sécurité



DevTools

- Pour que le devtools fonctionne en remote, vous devez le packager avec le livrable (ajuster le spring-boot-maven-plugin):

```
<build>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
      <configuration>
        <includeOptional>true</includeOptional>
        <excludeDevtools>false</excludeDevtools>
      </configuration>
    </plugin>
  </plugins>
</build>
```



DevTools

- Côté client (dans votre IDE), en restant dans votre projet
 - lancez la classe RemoteSpringApplication
 - Passez l'URL où est déployé votre projet en argument de lancement.
- Tout changement côté client, entraînera un redéploiement côté serveur
 - Vous pouvez influencer le temps entre chaque déploiement via
 - ❑ `spring.devtools.restart.poll-interval`
 - ❑ `spring.devtools.restart.quiet-period`



Maven

➤ Les dépendances Spring Boot

```
<dependencies>
```

```
<dependency>
```

```
<groupId>org.springframework.boot</groupId>
```

```
<artifactId>spring-boot-starter</artifactId>
```

```
</dependency>
```

Indispensable

```
<dependency>
```

```
<groupId>org.springframework.boot</groupId>
```

```
<artifactId>spring-boot-xxx-yyy</artifactId>
```

```
</dependency>
```

A vous de voir les
différents éléments dont
vous avez besoin

```
<!-- Pour nos tests unitaires -->
```

```
<dependency>
```

```
<groupId>org.springframework.boot</groupId>
```

```
<artifactId>spring-boot-starter-test</artifactId>
```

```
<scope>test</scope>
```

```
</dependency>
```

```
</dependencies>
```

Exemple : pour faire des
tests en Spring + JUnit



Maven

- Depuis Spring Boot 2.0, vous pouvez ajouter une dépendance pour vous aider lors de vos migrations
 - ➡ Par exemple de Spring Boot 2.x.x vers 4.y.x
- Cette dépendance vous indiquera les changements dans les clefs de configuration de Spring Boot (et c'est déjà pas mal)
 - ➡ En aucune manière elle ne changera votre code !
 - ❑ Il vous restera donc du boulot à faire, j'espère que vos testes unitaire sont à jours

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-properties-migrator</artifactId>
  <scope>runtime</scope>
</dependency>
```



Maven

- Les dépendances Spring Boot peuvent varier en fonction de vos besoins
 - Changement du log
 - Ajout d'une librairie
 - Besoin d'un serveur
 - ...
- Pour chaque besoin, une solution :
 - <https://docs.spring.io/spring-boot/reference/using/build-systems.html#using.build-systems.starters>



Maven

- `spring-boot-starter` : Core starter, including auto-configuration support, logging and YAML
- `spring-boot-starter-activemq` : Starter for using Apache ActiveMQ and JMS
- `spring-boot-starter-activemq-test` : Starter for testing using Apache ActiveMQ and JMS
- `spring-boot-starter-actuator-test` : Starter for testing Spring Boot's Actuator which provides production ready features to help you monitor and manage your application
- `spring-boot-starter-amqp` : Starter for using Spring AMQP and Rabbit MQ
- `spring-boot-starter-amqp-test` : Starter for testing Spring AMQP and Rabbit MQ
- `spring-boot-starter-artemis` : Starter for using Apache Artemis and JMS
- `spring-boot-starter-artemis-test` : Starter for testing Apache Artemis and JMS
- `spring-boot-starter-aspectj` : Starter for using aspect-oriented programming with AspectJ
- `spring-boot-starter-aspectj-test` : Starter for testing aspect-oriented programming with AspectJ
- `spring-boot-starter-batch` : Starter for using Spring Batch
- `spring-boot-starter-batch-jdbc` : Starter for using Spring Batch with JDBC
- `spring-boot-starter-batch-jdbc-test` : Starter for testing using Spring Batch with JDBC
- `spring-boot-starter-batch-test` : Starter for testing using Spring Batch
- `spring-boot-starter-cache` : Starter for using Spring's caching support
- `spring-boot-starter-cache-test` : Starter for testing Spring's caching support



Maven

- `spring-boot-starter-cassandra` : Starter for using Cassandra distributed database
- `spring-boot-starter-cassandra-test` : Starter for testing Cassandra distributed database
- `spring-boot-starter-classic` : Core classic starter, including full auto-configuration support, logging and YAML
- `spring-boot-starter-cloudfoundry` : Starter for using Cloud Foundry
- `spring-boot-starter-cloudfoundry-test` : Starter for testing Cloud Foundry
- `spring-boot-starter-couchbase` : Starter for using Couchbase document-oriented database
- `spring-boot-starter-couchbase-test` : Starter for testing Couchbase document-oriented database
- `spring-boot-starter-data-cassandra` : Starter for using Cassandra distributed database and Spring Data Cassandra
- `spring-boot-starter-data-cassandra-reactive` : Starter for using Cassandra distributed database and Spring Data Cassandra Reactive
- `spring-boot-starter-data-cassandra-reactive-test` : Starter for testing Cassandra distributed database and Spring Data Cassandra Reactive
- `spring-boot-starter-data-cassandra-test` : Starter for testing Cassandra distributed database and Spring Data Cassandra
- `spring-boot-starter-data-couchbase` : Starter for using Couchbase document-oriented database and Spring Data Couchbase
- `spring-boot-starter-data-couchbase-reactive` : Starter for using Couchbase document-oriented database and Spring Data Couchbase Reactive
- `spring-boot-starter-data-couchbase-reactive-test` : Starter for testing Couchbase document-oriented database and Spring Data Couchbase Reactive
- `spring-boot-starter-data-couchbase-test` : Starter for testing Couchbase document-oriented database and Spring Data Couchbase
- `spring-boot-starter-data-elasticsearch` : Starter for using Elasticsearch search and analytics engine and Spring Data Elasticsearch
- `spring-boot-starter-data-elasticsearch-test` : Starter for testing Elasticsearch search and analytics engine and Spring Data Elasticsearch



Maven

- **spring-boot-starter-webflux** : Starter for building WebFlux applications using Spring Framework's Reactive Web support
- **spring-boot-starter-websocket** : Starter for building WebSocket applications using Spring Framework's WebSocket support
- Liste non exhaustive des *starters*
 - <https://docs.spring.io/spring-boot/reference/using/build-systems.html#using.build-systems.starters>



Maven

➤ Exemple, utiliser Log4J2 à la place de Slf4J

```
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter</artifactId>
    <!-- On veut utiliser log4j2, donc on retire le module de base -->
    <exclusions>
      <exclusion>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-logging</artifactId>
      </exclusion>
    </exclusions>
  </dependency>

  <!-- Puis on ajoute la dépendance sur le module de Log4j2 -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-log4j2</artifactId>
  </dependency>
```

A indiquer sur tous
les spring-boot-
starter-xxx

A n'ajouter qu'une
seule fois

Spring Initializr

51

➤ Vous pouvez demander de l'aide à l'outil en ligne de Spring

➡ <https://start.spring.io/>

➤ Il vous permet de choisir les éléments dont vous avez besoin et vous fera un projet

The screenshot shows the Spring Initializr web interface at <https://start.spring.io/index.html>. The interface is divided into several sections for configuring a new Spring project:

- Project:** Includes radio buttons for **Gradle - Groovy** (selected), **Gradle - Kotlin**, and **Maven**.
- Language:** Includes radio buttons for **Java** (selected), **Kotlin**, and **Groovy**.
- Spring Boot:** Includes radio buttons for **4.1.1 (SNAPSHOT)**, **4.1.0** (selected), **4.0.8 (SNAPSHOT)**, **4.0.7**, and **3.5.16**.
- Project Metadata:** Includes text input fields for **Group** (com.example), **Artifact** (demo), and **Package name** (com.example.demo).
- Packaging:** Includes radio buttons for **Jar** (selected) and **War**.
- Configuration:** Includes radio buttons for **Properties** (selected) and **YAML**.
- Java:** Includes radio buttons for **26**, **25**, **21**, and **17** (selected).
- Dependencies:** A section with a button **ADD DEPENDENCIES... CTRL + B** and the text *No dependency selected*.

At the bottom of the interface, there are three buttons: **GENERATE CTRL + G**, **EXPLORE CTRL + SPACE**, and a button with three dots **...**.



La classe de configuration

- Une classe de configuration Spring Boot
 - ➡ Sera annotée en **@SpringBootApplication**
 - ❑ Annotation qui hérite de @Configuration
 - ❑ Elle peut charger d'autres classes en @Configuration
 - Vous pouvez faire usage de @Import(MaClasseConf.class)
 - Mais en principe c'est automatique car elle porte un component scan
 - ➡ Peut hériter ou implémenter des interfaces/classes spécifiques en fonction des besoins
 - ➡ Aura une méthode *main* qui s'occupera d'*auto monter* <=> de démarrer le Spring



53

La classe de configuration

➤ Exemple

```
package com.exo;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.ApplicationContext;

@SpringBootApplication
public class SpringBootConfiguration {

    public static void main(String[] args) {
        ApplicationContext appContext = SpringApplication.run(SpringBootConfiguration.class, args);

        MonBean unBean = appContext.getBean(MonBean.class);
    }
}
```

Enclenche automatiquement un
`@ComponentScan("lePackageOùCeTrouveCetteClasse")`



➤ Résultat du lancement

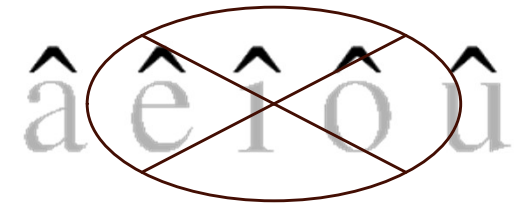
© Ferret Renaud



La classe de configuration

- Vous pouvez la paramétrer
 - Directement dans votre code Java
 - Via un fichier de configuration (meilleure solution)
 - ❑ `application.properties` ou `application.yml`

➤ ! ATTENTION !



- Le fichier properties / YML **ne doit pas contenir d'accents**
 - ❑ Même dans les commentaires, sinon, Maven ne fonctionnera plus
- Pensez à vérifier que votre fichier de configuration est bien en UTF-8



Fichier de configuration

- Le fichier pourra se trouver dans (par ordre de recherche)
 - Un dossier enfant **/config** se trouvant là où s'exécute le code
 - Dans le dossier où s'exécute le code
 - Dans un dossier **/config** présent au niveau du package
 - Dans le classpath
- Vous pouvez aussi indiquer explicitement où se trouve le fichier via les variables
 - **spring.config.name** et **spring.config.location**

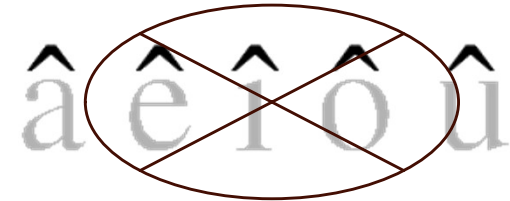


Fichier de configuration

➤ ! ATTENTION ! - Rappel

- Le fichier properties ne doit pas contenir d'accents (même dans les commentaires)

❑ Sinon, Maven ne fonctionnera plus



➤ Exemples d'utilisation d'un fichier spécifique de configuration

```
java -jar monProjetSpringBoot.jar --spring.config.name=monFichierDeconf
```

```
java -jar monProjetSpringBoot.jar  
--spring.config.location=classpath:/x/y/z/default.properties,classpath:/x/y/oubien.properties
```



Fichier de configuration

- En Maven, votre fichier de configuration pour Spring Boot se trouvera classiquement dans **src/main/resources/** ou **src/test/resources/**
- Le contenu du fichier sera représentatif des contraintes de votre projet
- On peut y retrouver des informations techniques
 - Numéro de port
 - Login / mot de passe
 - Options à destination des classes techniques



Fichier de configuration

- Exemple de fichier properties qui va désactiver la bannière du Spring Boot

```
# Exemple de fichier application.properties  
spring.main.banner-mode=off
```

- Vous pouvez y faire usage de variables

```
ma.variable=Bonjour  
utilisation.de.ma.variable=${ma.variable} tout le monde
```

- Vous pouvez y faire usage de tableaux

```
ma.variable[0]=premiere valeur  
ma.variable[1]=seconde valeur
```



Fichier de configuration

➤ Vous pouvez y faire usage de valeurs aléatoires (via le SpEL)

```
#random int
app.location-x=${random.int}
app.location-y=${random.int}

#random int with max
app.user-age=${random.int(100)}

#random int range
app.max-users=${random.int[1,10000]}
```

```
#random long with max
app.refresh-rate-milli=${random.long(1000000)}

#random long range
app.initial-delay-
milli=${random.long[100,9000000000000000000]}

#random 32 bytes
app.user-password=${random.value}

#random uuid. Uses java.util.UUID.randomUUID()
app.instance-id=${random.uuid}
```



Fichier de configuration - Profile

- Vous pouvez faire usage de profils
 - Via le nom des fichiers de configuration
 - Et/ou via l'annotation **@Profile**
 - Avec, en complément l'utilisation des annotations conditionnelles
- Exemple
 - Un fichier **application.properties** : chargé à chaque fois
 - Un fichier **application-prod.properties** : chargé si le profil **prod** est activé
 - Un fichier **application-prex.properties** : chargé si le profil **prex** est activé
 - Un fichier **application-test.properties** : chargé si le profil **test** est activé



Fichier de configuration - Profile

➤ Pour activer un (ou plusieurs) profil(s) :

➤ Dans le fichier principal

❑ `spring.profiles.active=dev,test,prex`

Activera les 3 profils

➤ Dans les options de lancement

❑ `java -Dspring.profiles.active=prod -jar mon.jar`

❑ `java -jar mon.jar --spring.profiles.active=prod`

➤ En tant que variable Système

❑ `SPRING_PROFILES_ACTIVE=prod`

➤ Dans le code de classe Spring Boot

❑ En utilisant la méthode `setAdditionalProfiles` de la classe `SpringApplication`

➤ Pour les tests (et uniquement les tests) via l'annotation

❑ `@ActiveProfiles("test")`



Profile - Rappel

- Dans votre code Java, l'annotation **@Profile** permet d'indiquer que le **@Bean** n'est valable que pour le profil indiqué

```
@Configuration
public class SpringConfigurationData {

    @Profile({ "test", "dev" })
    @Bean
    public TomcatZContainerFactory tomcatFactory() {
        // ...
    }
}
```

Existe pour le profil
"test" et le "dev"

```
@Service
@Profile("test")
public class TestService implements IMonService {

    @Override
    public String faire() {
        return "Test!";
    }
}
```

Existe pour le profil
"test"

```
@Service
@Profile("prod")
public class ProdService implements IMonService {

    @Override
    public String faire() {
        return "Prod!";
    }
}
```

Existe pour le profil
"prod"



Fichier de configuration - Profile

- Depuis Spring Boot 2.4, vous pouvez aussi regrouper des profils en groupes de profils

- **Exemple de groupes**

```
spring.profiles.group.prod=profilA, profilB  
spring.profiles.group.dev=profilA, profilC  
  
spring.profiles.active=prod
```




Fichier de configuration - Profile

➤ Les clefs liées aux profiles

- `spring.profiles.active` : les profils à activer
- `spring.profiles.default` : les profils à prendre si aucun n'est spécifié explicitement (par *active*)
- `spring.profiles.include` : les profils à charger quoi qu'il en soit

➤ **Remarque** : Tentez toujours de rester simple dans votre mécanique de profils



Profile Maven & Spring

- Un profile Maven n'a rien à voir avec un profile Spring

```
<profiles>
  <profile>
    <id>postgresql</id>
    <dependencies>
      <dependency>
        <groupId>org.postgresql</groupId>
        <artifactId>postgresql</artifactId>
        <scope>runtime</scope>
      </dependency>
    </dependencies>
  </profile>
</profiles>
```

```
mvn spring-boot:run -Ppostgresql -Dspring-
boot.run.profiles=prod
```

- Cependant, via la propriété Maven `<spring-boot.run.profiles>` vous pouvez faire passer le nom du/des profiles Spring que vous voulez à partir de Maven



Profile Maven & Spring

- Exemple de passage d'un profile Maven vers un profile Spring

```
<profile>
  <id>prod</id>
  <properties>
    <spring-boot.run.profiles>prod</spring-boot.run.profiles>
  </properties>
</profile>
```

```
mvn spring-boot:run -Ppostgresql
```

- Enclenchera automatiquement le profile Spring prod sans que vous n'ayez à écrire explicitement l'information



Valeur d'une clef

➤ Vous pouvez écraser toutes les informations du properties en spécifiant sa clef

➤ `--nom.de.clef` : c'est spécifique à Spring Boot

➤ On le place **APRES** le `-jar mon.jar`

```
java -jar monProjetSpringBoot.jar --ma.clef.dans.mon.properties=maNouvelleValeur
```

➤ Ou,

➤ `-Dnom.de.clef` : c'est du Java, cela passe l'information au niveau propriété système

➤ Toujours **AVANT** le `-jar mon.jar`

```
java -Dma.clef.dans.mon.properties=maNouvelleValeur -jar monProjetSpringBoot.jar
```



Travaux pratiques

69

Réaliser les travaux pratiques suivants:

TP 01 : Un projet simple sous Spring Boot



Spring Boot et les tests Unitaires

- Très simple, il suffit d'ajouter l'annotation **@SpringBootTest** à votre classe de test unitaire
 - **JUnit 4** uniquement pour **Spring 4.x**
 - Possibilité d'utiliser **JUnit 5** en Spring **5.2.x** = Par défaut à partir de Spring **Boot 2.2.x**
- Elle chargera votre classe annotée **@SpringBootApplication**
 - Il faudra parfois la cibler directement via son attribut **classes**
- N'oubliez pas d'ajouter la dépendance vers spring-boot-starter-test dans votre pom.xml

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-test</artifactId>
  <scope>test</scope>
</dependency>
```



Spring Boot et les tests Unitaires

- Attention : **depuis Spring Boot 4.xx.yy**
 - chaque starter possède son propre starter de test.
 - Nous sommes en **JUnit 6.x**
- Par exemple, si vous voulez faire du Kafka, vous ajouterez

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-kafka</artifactId>
</dependency>
```

- Si vous avez besoin de réaliser des tests unitaires dans votre projet sur Kafka il faudra aussi ajouter

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-kafka-test</artifactId>
</dependency>
```



Spring Boot et les tests Unitaires

➤ Exemple

```
@SpringBootTest
class MonBeanTest {

    @Autowired
    private MonBean monBean;

    @Test
    void testMe() {
        // ...
    }
}
```

Attention à
vos
imports

La visibilité d'une classe
de test en JUnit 5/6 doit
être package

La visibilité d'une
méthode de test en
JUnit 5/6 doit être
package



Travaux pratiques

73

Réaliser les travaux pratiques suivants:

TP 08 suite : ajoutez votre projet et vos tests unitaires



Spring Boot versions

74

➤ C'est le Spring Boot qui gère les **versions** de vos dépendances

➤ 9x10 elles sont à jour, il suffit d'ajouter l'artefactId et le groupId

➤ Pour connaître leurs valeurs, il suffit d'aller voir le **POM effectif**

```
exo03.spring.boot.jpa/pom.xml 35
36 <url>https://github.com/spring-projects/spring-boot/spring-boot-starter-parent/exo03.spring.boot.jpa</url>
37 </scm>
38 <properties>
39 <spring-batch.version>3.0.9.RELEASE</spring-batch.version>
40 <undertow.version>1.4.25.Final</undertow.version>
41 <httpClient.version>4.5.5</httpClient.version>
42 <maven-surefire-plugin.version>2.18.1</maven-surefire-plugin.version>
43 <webjars-locator.version>0.32-1</webjars-locator.version>
44 <spring-ldap.version>2.3.2.RELEASE</spring-ldap.version>
45 <spring-security-oauth.version>2.0.15.RELEASE</spring-security-oauth.version>
46 <hibernate.version>5.2.17.Final</hibernate.version>
47 <groovy.version>2.4.15</groovy.version>
48 <spring-social-linkedin.version>1.0.2.RELEASE</spring-social-linkedin.version>
49 <jmustache.version>1.13</jmustache.version>
50 <spring-cloud-connectors.version>1.2.5.RELEASE</spring-cloud-connectors.version>
51 <mockito.version>1.10.19</mockito.version>
52 <json.version>20140107</json.version>
53 <maven-antrun-plugin.version>1.8</maven-antrun-plugin.version>
54 <java.version>1.6</java.version>
55 <bitronix.version>2.1.4</bitronix.version>
56 <commons-beanutils.version>1.9.3</commons-beanutils.version>
57 <exec-maven-plugin.version>1.5.0</exec-maven-plugin.version>
58 <atomikos.version>3.9.3</atomikos.version>
59 <gradle.version>2.9</gradle.version>
60 <maven-javadoc-plugin.version>2.10.4</maven-javadoc-plugin.version>
61 <jersey.version>2.25.1</jersey.version>
62 <jaxen.version>1.1.6</jaxen.version>
63 <caffeine.version>2.3.5</caffeine.version>
64 <couchbase-cache-client.version>2.1.0</couchbase-cache-client.version>
65 <sun-mail.version>1.5.6</sun-mail.version>
66 </properties>
67 </parent>
68 </pom>
```

Overview Dependencies Dependency Hierarchy **Effective POM** pom.xml



Spring Boot versions

- Une fois la clef trouvée dans le pom effectif, ajoutez cette même clef dans votre pom avec la valeur désirée

```
<properties>
  <maven.compiler.source>1.8</maven.compiler.source>
  <maven.compiler.target>1.8</maven.compiler.target>
  <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  <project.reporting.outputEncoding>UTF-8</project.reporting.outputEncoding>

  <!--
  On surcharge la propriete du Spring Boot que l'on peut
  trouver en allant dans l'onglet Effective POM de Eclipse
  -->
  <hibernate.version>5.3.7.Final</hibernate.version>
  <log4j2.version>2.11.2</log4j2.version>
  <mysql.version>8.0.13</mysql.version>
</properties>
```



Spring Boot et JDBC - Maven

- Il suffira d'ajouter la dépendance vers spring-boot-starter-jdbc

```
<dependency>  
  <groupId>org.springframework.boot</groupId>  
  <artifactId>spring-boot-starter-jdbc</artifactId>  
</dependency>
```

- Vous ajouterez aussi les dépendances vers votre driver JDBC
 - MySQL, Oracle, ...



Spring Boot et JDBC - Properties

77

- Dans le fichier properties, il faudra indiquer les éléments de mapping associés à la DataSource

Pour laisser le Spring Boot monter une DataSource

```
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
spring.datasource.url=jdbc:mysql://localhost/banque?useSSL=false&serverTimezone=Europe/Paris&allowPublicKeyRetrieval=true
spring.datasource.username=root
spring.datasource.password=
```

Exemple parametrage du pool (en pool HikariCP par default en Spring Boot 2/3/4)

```
# Les temps sont en millisecondes
spring.datasource.hikari.connection-timeout=20000
spring.datasource.hikari.minimum-idle=15
spring.datasource.hikari.maximum-pool-size=50
spring.datasource.hikari.idle-timeout=300000
spring.datasource.hikari.max-lifetime=1200000
```

Parametrage du pool (en pool tomcat par default en Spring Boot 1.x)

```
# https://tomcat.apache.org/tomcat-8.5-doc/jdbc-pool.html#Common_Attributes
spring.datasource.tomcat.max-wait=20000
spring.datasource.tomcat.max-active=50
spring.datasource.tomcat.max-idle=20
spring.datasource.tomcat.min-idle=15
```



Spring Boot et JDBC - Properties

78

➤ Si vous ne souhaitez pas faire usage du pool tomcat, vous devrez le préciser dans Maven et le fichier de properties

Exemple En DBCP2

spring.datasource.dbcp2.initial-size=7

spring.datasource.dbcp2.max-total=20

spring.datasource.dbcp2.pool-prepared-statements=true

```
<properties>
  <commons-dbcp2.version>2.3.0</commons-dbcp2.version>
</properties>

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter</artifactId>
  <exclusions>
    <!-- On veut utiliser DBCP2, donc on retire le module de base -->
    <exclusion>
      <groupId>org.apache.tomcat</groupId>
      <artifactId>tomcat-jdbc</artifactId>
    </exclusion>
  </exclusions>
</dependency>

<dependency>
  <groupId>org.apache.commons</groupId>
  <artifactId>commons-dbcp2</artifactId>
</dependency>
```



Spring Boot et les Pools

➤ En Spring Boot 1 : Pool Tomcat par défaut

```
spring.datasource.tomcat.max-wait=20000  
spring.datasource.tomcat.max-active=50  
spring.datasource.tomcat.max-idle=20  
spring.datasource.tomcat.min-idle=15
```

➤ En Spring Boot 2/3/4 : Pool Hikari par défaut

```
spring.datasource.hikari.connection-timeout=20000  
spring.datasource.hikari.minimum-idle=15  
spring.datasource.hikari.maximum-pool-size=50  
spring.datasource.hikari.idle-timeout=300000  
spring.datasource.hikari.max-lifetime=1200000
```




Travaux pratiques

80

Réaliser les travaux pratiques suivants, dans un contexte non web :

TP 10 : faites usage du Spring Boot Starter JDBC



Spring Boot et JPA - Maven

- Il suffira d'ajouter la dépendance vers spring-boot-starter-data-jpa

```
<dependency>  
  <groupId>org.springframework.boot</groupId>  
  <artifactId>spring-boot-starter-data-jpa</artifactId>  
</dependency>
```

- Vous ajouterez aussi les dépendances vers votre implémentation de JPA
 - Hibernate par exemple



Spring Boot et JPA - Properties

- Dans le fichier properties, il faudra indiquer
 - les éléments de mapping associés à la DataSource
 - les éléments de configurations associés à JPA

```
# Pour gerer le lazy
spring.jpa.open-in-view=true
# Afin que JPA respecte l'annotation @Column des @Entity il est impératif d'ajouter :
spring.jpa.hibernate.naming.implicit-strategy=org.hibernate.boot.model.naming.ImplicitNamingStrategyLegacyJpaImpl
spring.jpa.hibernate.naming.physical-strategy=org.hibernate.boot.model.naming.PhysicalNamingStrategyStandardImpl
# On ne veut pas generer la base a partir du code
spring.jpa.generate-ddl=false
spring.jpa.hibernate.ddl-auto=none
# Proprietes Hibernates
spring.jpa.properties.hibernate.dialect.storage_engine=innodb
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQLDialect
spring.jpa.properties.hibernate.connection.handling_mode=DELAYED_ACQUISITION_AND_HOLD
spring.jpa.properties.hibernate.cache.provider_class=org.hibernate.cache.NoCacheProvider
spring.jpa.properties.hibernate.cache.use_second_level_cache=false
# Si vous voulez faire du AUTO dans @Id
spring.jpa.properties.hibernate.id.new_generator_mappings=false
```



Travaux pratiques

83

Réaliser les travaux pratiques suivants, dans un contexte non web :

TP 13 : faites usage du Spring Boot Starter JPA avec Hibernate 5.3 (et non pas 5.0)



Spring Boot et le web

➤ Avec Spring Boot :

➡ Quand je **développe**, je vais pouvoir

- ❑ **Déployer** sur un Tomcat (comme avant sans Spring Boot)

- ❑ **Lancer** mon application Spring Boot qui lancera un serveur JEE embarqué

➡ Quand je **livre**, je vais pouvoir livrer un WAR/JAR/Image

- ❑ Un WAR : Normal/classique (comme avant sans Spring Boot)

- ❑ Un WAR/JAR : Auto exécutable via le plugin Maven de Spring Boot

- ❑ Une Image Docker native ou non



Spring Boot et le web - Maven

- La dépendance **spring-boot-starter-web** remplace la dépendance **spring-boot-starter**

```
<!-- Pour les pages JSP -->
<dependency>
  <groupId>org.apache.tomcat.embed</groupId>
  <artifactId>tomcat-embed-jasper</artifactId>
  <scope>provided</scope>
</dependency>
```

Si vous avez des JSP
Va récupérer le javaee-web-api

```
<!-- Pour avoir un WAR avec notre Spring Boot -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
  <!-- On veut utiliser log4j2, donc on retire le module de base -->
  <exclusions>
    <exclusion>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-logging</artifactId>
    </exclusion>
  </exclusions>
</dependency>
```

Remplace ~~spring-boot-starter~~

Si on ne veut pas de Log4J



Spring Boot et le web - Maven

- **Attention** en Spring Boot 4, la dépendance a changé de nom : spring-boot-starter-**webmvc**

```
<!-- En Spring Boot 4+ -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-webmvc</artifactId>
</dependency>
```

- De même, pour les tests, on devra ajouter

```
<!-- En Spring Boot 4+ -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-webmvc-test</artifactId>
  <scope>test</scope>
</dependency>
```



Spring Boot et le web - Maven

- En plus des dépendances Spring Boot, vous n'oublierez pas
 - Dépendances vers JSTL
 - Data Source / Pool
 - Tag Libs ...
- **Attention** à votre version JEE et aux versions minimum associées à votre version de Spring Boot
 - Rappel : En Spring Boot 4 <=> Jakarta EE 11
- N'hésitez pas à consulter le "Effective POM"
 - Afin de voir les dépendances présentes et leurs versions



Spring Boot et le web - Serveurs

- La dépendance `spring-boot-starter-web` va récupérer la dépendance `spring-boot-starter-tomcat`
- Vous pouvez faire usage d'autre serveur si vous voulez
 - `spring-boot-starter-jetty`
 - `spring-boot-starter-undertow`
- La configuration du serveur peut se faire via le fichier `application.properties`
 - Toutes les options sont visibles dans la classe `org.springframework.boot.autoconfigure.web.ServerProperties`



Spring Boot et le web - Classe Java

- Si vous voulez faire usage d'un WAR classique <=> déployable sur un serveur JEE
 - votre classe Spring Boot hérite de **SpringBootServletInitializer**
 - vous avez une classe qui hérite de **SpringBootServletInitializer** (qui n'est pas forcément celle annotée en `@SpringBootApplication`)
 - ❑ Elle implémente la méthode **configure**
- Sans la classe **SpringBootServletInitializer**
 - Votre WAR **ne sera pas** déployable sur un serveur JEE classique
 - ❑ Il reste auto-exécutable

Spring Boot et le web - Classe Java

- Exemple, dans le cas où vous voulez faire un WAR classique (union des deux classes de configurations)

@SpringBootApplication

```
public class SpringBootConfiguration extends SpringBootServletInitializer {
```

@Override

```
protected SpringApplicationBuilder configure(SpringApplicationBuilder application) {  
    return application.sources(SpringBootConfiguration.class);  
}
```

```
public static void main(String[] args) {
```

```
    SpringApplication springApplication = new SpringApplication(SpringBootConfiguration.class);  
    springApplication.setBannerMode(Banner.Mode.OFF);  
    springApplication.setLogStartupInfo(false);  
    springApplication.run(args);
```

```
}
```

```
}
```

Exécuté quand
déployé sur un
Serveur JEE

Exécuté quand
lancé en mode
auto-exécutable

Spring Boot et le web - Fichier properties



- Le fichier `application.properties` pourra définir
 - les propriétés du serveur (port, SSL ...)
 - Le Context Root/Path
 - ❑ `server.contextPath=/en.spring.boot.version.un`
 - ❑ `server.servlet.context-path=/en.spring.boot.version.deux+`
- Par défaut le context root vaut '/'



Spring Boot et le web

- Si vous lancez votre classe Spring Boot
 - C'est elle qui va lancer le serveur JEE embarqué via la méthode main
 - C'est elle qui va y placer votre WAR
 - C'est elle qui va charger le Spring (via la méthode main)
 - C'est à vous de programmer vos éléments JNDI
 - ❑ DataSource JEE et autres
- Sinon, en déploiement manuel sur un serveur JEE
 - C'est à vous de lancer le serveur JEE
 - C'est à vous d'y déployer votre WAR
 - C'est le Spring Boot qui va gérer le Spring (via la méthode configure)
 - C'est le serveur JEE qui gère ses éléments JNDI
 - ❑ DataSource JEE et autres



Travaux pratiques

93

Réaliser les travaux pratiques suivants, dans un contexte web :

TP 14 : faites usage du Spring Boot Starter Web



Spring Boot Security - Maven

- Il suffira d'ajouter la dépendance vers spring-boot-starter-security

```
<dependency>  
  <groupId>org.springframework.boot</groupId>  
  <artifactId>spring-boot-starter-security</artifactId>  
</dependency>
```

- Vous ajouterez aussi les dépendances vers certains modules de Spring Security
 - spring-security-taglibs par exemple



Spring Boot Security - Classe / Properties

95

- Il n'y a rien de particulier à préciser dans la classe Spring Boot
- Il n'y a rien de particulier à préciser dans le fichier `application.properties`
 - Vous pouvez au pire indiquer un login/password par défaut mais c'est une mauvaise idée
- La classe de configuration Spring Security
 - Ne bouge pas d'un pouce (comparativement aux autres modes de configuration)
 - ☐ Attention : elle peut tout de même changer si vous migrez de Spring Boot
 - Elle sera montée par la classe Spring Boot automatiquement



Travaux pratiques

96

Réaliser les travaux pratiques suivants, dans un contexte web :

TP 19 : faites usage du Spring Boot Starter Security



Spring Boot - Divers

- Dans certain cas, vous ne voudrez pas que Spring Boot s'occupe de fabriquer tel ou tel objet
 - Par exemple, si vous faites de l'Hibernate sans JPA
- Il suffira d'indiquer à Spring Boot de ne pas s'occuper de telle ou telle classe
 - Utilisation de l'attribut **exclude** de l'annotation **SpringBootApplication**

```
@SpringBootApplication(exclude = { DataSourceAutoConfiguration.class,  
JpaRepositoriesAutoConfiguration.class, HibernateJpaAutoConfiguration.class })
```



Spring Boot - Divers

- La problématique classique est souvent de retrouver les clefs nécessaires à Spring Boot
- Elles sont regroupées ici (Spring Boot 3 et 4) :
 - ➡ <https://docs.spring.io/spring-boot/appendix/application-properties/index.html>
- Attention : **à votre version** de Spring Boot et aux noms des clefs
 - ➡ Clefs pour Spring Boot 1 : <https://docs.spring.io/spring-boot/docs/1.5.x/reference/html/common-application-properties.html>



Spring Boot - Les Filter JEE

- En JEE un *filter* est annoté en **@WebFilter** et cible des URLs
- En Spring Boot
 - Si le *filter* cible '/*', remplacez simplement **@WebFilter** par **@Component** et vérifiez qu'il est dans un **@ComponentScan**
 - Sinon, gardez votre **@WebFilter**, ajoutez un **@ServletComponentScan** sur votre classe **@SpringBootApplication**



Spring Boot - Les Listener JEE

- En JEE un *listener* est annoté en **@WebListener** et implémente une ou plusieurs interfaces liées aux événements JEE
- En Spring Boot
 - Remplacez simplement **@WebListener** par **@Component** et vérifiez qu'il est dans un **@ComponentScan**



Spring Boot - Les Servlet JEE

- En JEE une Servlet est annoté en **@WebServlet** et répond à des URLs

- En Spring Boot
 - Remplacez simplement **@WebServlet** par **@Controller** et vérifiez qu'il est dans un **@ComponentScan**
 - ❑ Ensuite, utilisez l'annotation **@RequestMapping** ou **@GetMapping**, **@PostMapping**, ... pour gérer les URLs
 - Sinon, gardez votre **@WebServlet**, ajoutez un **@ServletComponentScan** sur votre classe **@SpringBootApplication**

Spring Boot - Annotations conditionnelles



- Spring Boot met à disposition des annotations conditionnelles
- Avec, vous pouvez dire de charger tel bean si tel autre est présent ou absent par exemple
- Pratique quand on fait usage des profiles



Spring Boot - Annotations conditionnelles

103

- **@ConditionalOnClass({A.class, B.class})**
 - Charger le @Bean ou le @Configuration si les classes A et B sont présentes
- **@ConditionalOnMissingClass("a.b.c.A")**
 - Charger le @Bean ou le @Configuration si la classe A est absente
- **@ConditionalOnBean(A.class)**
 - Charger le @Bean ou le @Configuration si un bean de type A est présent
- **@ConditionalOnMissingBean(A.class)**
 - Charger le @Bean ou le @Configuration si aucun bean de type A n'est présent



Spring Boot - Annotations conditionnelles

104

- **@ConditionalOnProperty**(name="server.port", havingValue="8000")
 - Charger le @Bean ou le @Configuration si la propriété server.port est définie et porte la valeur 8000
- **@ConditionalOnProperty**(name="server.port", matchIfMissing=true)
 - Charger le @Bean ou le @Configuration si la propriété server.port est absente
- **@ConditionalOnResource**(resources = "classpath:spring/fichier.xml")
 - Charger le @Bean ou le @Configuration si un fichier "spring/fichier.xml" est présent
- **@ConditionalOnExpression**("monexpressionSpEL")
 - Charger le @Bean ou le @Configuration si l'expression SpEL renvoie true

Spring Boot - Annotations conditionnelles



➤ **@ConditionalOnWebApplication**

- Charger le @Bean ou le @Configuration si le projet est une application Web

➤ **@ConditionalOnNotWebApplication**

- Charger le @Bean ou le @Configuration si le projet n'est pas une application Web



Spring Boot - Annotations conditionnelles

106

- **@ConditionalOnJava**(JavaVersion.SEVEN)
 - Charger le @Bean ou le @Configuration si le projet fait usage de la version du Java indiquée

- **@ConditionalOnCloudPlatform**(CloudPlatform.CLOUD_FOUNDRY)
 - Charger le @Bean ou le @Configuration si le projet fait usage de la CloudPlatform indiquée



Spring Boot - Annotations conditionnelles

107

- Si toutes ses annotations ne répondent pas à vos besoins vous pouvez réaliser les votre
 - En utilisant sur votre `@Bean` ou `@Configuration` l'annotation **@Conditional**
 - En réalisant une classe qui implémentera l'interface **Condition** du Spring
 - ❑ Et définir sa méthode **boolean matches(ConditionContext, AnnotatedTypeMetadata)**



Spring Boot - Annotations conditionnelles

108

➤ Exemple d'utilisation @Conditional

```
public class OnUnixCondition implements Condition {  
  
    @Override  
    public boolean matches(  
        ConditionContext context,  
        AnnotatedTypeMetadata metadata) {  
        return SystemUtils.IS_OS_LINUX;  
    }  
}
```

```
...  
@Bean  
@Conditional(OnUnixCondition.class)  
public UnixBean unixBean() {  
    return new UnixBean();  
}  
...
```